

Deep Learning for teens

By Hamid Sadeghi

Table of contents

- ▶ RNN
- ▶ Real life use case of RNN
- ▶ Why we can't use simple neural network for sequences problem
- ▶ Simple RNN Example
- ▶ All RNN Types
- ▶ Vanilla RNN
- ▶ Vanishing memory in RNN
- ▶ RNN Input-Output Types
- ▶ LSTM RNN
- ▶ GRU RNN
- ▶ Side-by-side comparison

- ▶ Homework

RNN

- ▶ Mainly use for NLP task

Real life use case of RNN

- ▶ In email it guess whats nest
- ▶ Translation. Translate from one language to another
- ▶ NER :name entity recognition
 - ▶ An entity is something in the real world that a word or phrase refers to.
- ▶ Sentiment analysis
 - ▶ is the task of figuring out whether a piece of text expresses a ****positive****, ****negative****, or ****neutral**** feeling.

Why we can't use simple neural network for sequences problem

- ▶ - ANN needs fixed input size, but sentences have different lengths.
- ▶ - One-hot vectors are huge → too much computation.
- ▶ - Turning a whole sentence into one number loses all meaning.
- ▶ - ANN does ****not**** share parameters across positions.
- ▶ - ANN cannot understand word order.
- ▶ - Sequence and context matter → ANN cannot handle them.

Simple RNN Example

- ▶ Task: predict next word in “I love”
- ▶ **Step 1**
 - ▶ Input: “I” Hidden state becomes memory of “I”.
- ▶ **Step 2**
 - ▶ Input: “love” Hidden state becomes memory of “I love”.
- ▶ **Step 3**
 - ▶ RNN predicts next word: → “you”
- ▶ That’s it.

All RNN Types

- ▶ **Vanilla RNN** → basic, simple memory
- ▶ **LSTM** → long-term memory, avoids vanishing gradient
- ▶ **GRU** → simpler than LSTM, similar performance
- ▶ **Fast RNN** → faster & more stable training than vanilla RNN
- ▶ **Bidirectional RNN** → reads sequence forward + backward
- ▶ **Encoder-Decoder RNN** → used for translation, seq2seq
- ▶ **Stacked RNN** → multiple RNN layers on top of each other

Vanilla RNN

- ▶ Hidden state
 - ▶ The hidden state is the RNN's short-term memory – it carries whatever the network remembers from the previous step
- ▶ How it works
 - ▶ Takes one input at a time (word, character, timestep).
 - ▶ Combines the new input with the previous hidden state.
 - ▶ Produces a new hidden state and an output.
 - ▶ **Vanilla RNN doesn't erase – it adds, but the old gets buried**

Vanishing memory in RNN

- ▶ **Vanishing memory** means:
 - ▶ As the sequence gets longer, a Vanilla RNN quickly forgets earlier information.
 - ▶ At t_1 , the RNN remembers the first word strongly
 - ▶ At t_5 , it remembers it weakly
 - ▶ At t_{20} , it basically forgets it completely
 - ▶ This happens because the hidden state keeps getting multiplied by weights at every time step. When those weights are small (less than 1), repeated multiplication makes the influence of earlier inputs shrink toward zero.

RNN Input-Output Types

► One-to-One

- Single input → single output
- Example: image → label

► One-to-Many

- Single input → sequence output
- Example: image → caption

► Many-to-One

- Sequence input → single output
- Example: sentence → sentiment

► Many-to-Many

- Sequence input → sequence output
- Example: translation

LSTM RNN

- ▶ LSTMs were created specifically to overcome the vanishing memory (vanishing gradient) problem in Vanilla RNNs.
- ▶ In RNN we use tanh as activation function
- ▶ imagine we're at time step 1 (T_1). In a Vanilla RNN, we'd have just one memory. But in an LSTM, we have both short-term (h_t) and long-term (c_t) memory going on.
- ▶ Long Short-Term Memory
- ▶ LSTM has 3 gates: forget, input, output

GRU RNN

- ▶ Gated Recurrent Unit
- ▶ GRU has 2 gates: update, reset

Side-by-side comparison

- ▶ LSTM is like a smart notebook with 3 switches:
 - ▶ What to forget
 - ▶ What to write
 - ▶ What to show
- ▶ GRU is like a smart sticky note with 2 switches:
 - ▶ How much old info to keep
 - ▶ How much past to forget

Homework

- ▶ I want you to use the `mini_sentiment.csv` Dataset and implement three models:
 - ▶ Vanilla RNN
 - ▶ LSTM RNN
 - ▶ GRU RNN
- ▶ Then calculate the accuracy for each model and compare the results.